Bias and Variance in Machine Learning - A Fantastic Guide for Beginners!

We use cookies essential for this site to function well. Please click to help us improve its usefulness with additional cookies. Learn about our use of cookies in our Privacy Policy & Cookies Policy.

Show details

Bias and Variance in Machine Learning – A Fantastic Guide for Beginners!

Last Updated: 07 Apr, 2025

Let us talk about the weather. It rains only if it's a little humid and does not rain if it's windy, hot or freezing. In this case, how would you train a predictive model and ensure that there are no errors in forecasting the weather? You may say that there are many learning algorithms to choose from. They are distinct in many ways but there is a major difference in what we expect and what the model predicts. That's the concept of Bias and Variance Tradeoff. In this article, you will get to know about the bias variance tradeoff, with bias and variance in machine learning, also you will get to know about What is bias and variance in machine learning.

Usually, Bias and Variance Tradeoff is taught through dense mathematical formulas. But in this article, I have attempted to explain Bias and Variance as simply as possible!

My focus will be to spin you through the process of understanding the problem statement and ensuring that you choose the best model where the Bias and Variance errors are minimal.

For this, I have taken up the popular Pima Indians Diabetes dataset. The dataset consists of diagnostic measurements of adult female patients of Native Indian Pima Heritage. For this dataset, we are going to focus on the "Outcome" variable – which indicates whether the patient has diabetes or not. Evidently, this is a binary classification problem and we are going to dive right in and learn how to go about it.

In this article, you will explore the bias-variance tradeoff in machine learning, learning how bias and variance affect model performance and the importance of balancing these two factors for optimal results.

If you are interested in this and data science concepts and want to learn practically refer to our course- Introduction to Data Science

Free Certification Courses

Data Science Bundle (4 Free Courses)

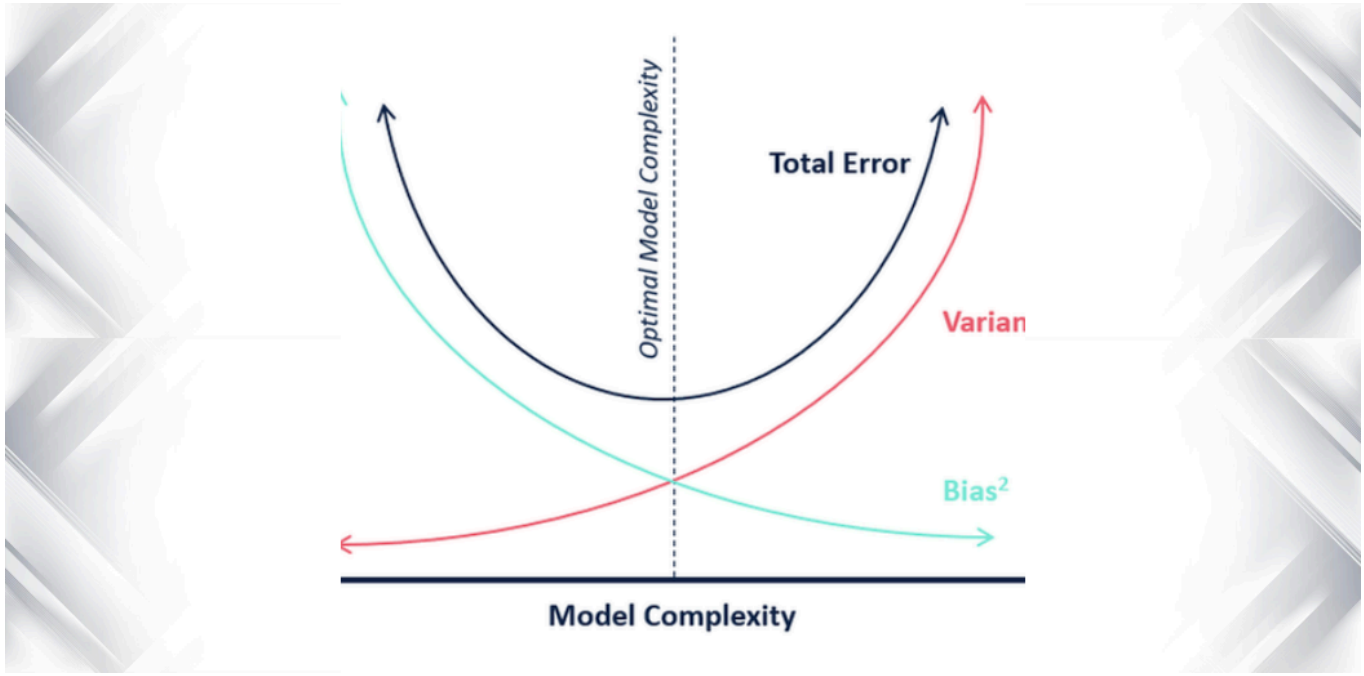
[

Python • Machine Learning • Deep Learning • NLP • Data Engineering • Hands-on Projects • Expert-led

](https://www.analyticsvidhya.com/courses/learning-path/data-science-program/?utm_source=blog&utm_medium=free_course_banner&utm_term=free_course_auto_enrollment)

Evaluating your Machine Learning Model

The primary aim of the Machine Learning models is to learn from the given data and generate predictions based on the pattern observed during the learning process. However, our task doesn't end there. We need to continuously make improvements to the models, based on the kind of results it generates. We also quantify the model's performance using metrics like Accuracy, Mean Squared Error(MSE), F1-Score, etc and try to improve these metrics. This can often get tricky when we have to maintain the flexibility of the model without compromising on its correctness.



Bias And Variance Tradeoff

A supervised Machine Learning model aims to train itself on the input variables(X) in such a way that the predicted values(Y) are as close to the actual values as possible (Modafinil). This difference between the actual values and predicted values is the **error** and it is used to evaluate the model. The error for any supervised Machine Learning algorithm comprises of 3 parts:

1. Bias error
2. Variance error
3. The noise

While the noise is the irreducible error that we cannot eliminate, the other two i.e. Bias and Variance are reducible errors that we can attempt to minimize as much as possible.

In the following sections, we will cover the Bias error, Variance error, and the Bias-Variance tradeoff which will aid us in the best model selection. And what's exciting is that we will cover some techniques to deal with these errors by using an example dataset.

Problem Statement and Primary Steps

As explained earlier, we have taken up the Pima Indians Diabetes dataset and formed a classification problem on it. Let's start by gauging the dataset and observe the kind of data we are dealing with. We will do this by importing the necessary libraries:

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import confusion_matrix
from sklearn import metrics
import matplotlib.pyplot as plt
%matplotlib inline
```

Now, we will load the data into a data frame and observe some rows to get insights into the data.

Python Code:

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import confusion_matrix
from sklearn import metrics
import matplotlib.pyplot as plt
#%matplotlib inline

data_file_path = 'diabetes.csv'
data_df = pd.read_csv(data_file_path)
print(data_df.head())
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

We need to predict the ‘Outcome’ column. Let us separate it and assign it to a target variable ‘y’. The rest of the data frame will be the set of input variables X.

```
y = data_df["Outcome"].values
x = data_df.drop(["Outcome"],axis=1)
```

Now let’s scale the predictor variables and then separate the training and the testing data.

```
from sklearn.preprocessing import StandardScaler
ss = StandardScaler()
data_df = ss.fit_transform(data_df)

#Divide into training and test data
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size = 0.3) #
70% training and 30% test
```

Since the outcomes are classified in a binary form, we will use the simplest K-nearest neighbor classifier(Knn) to classify whether the patient has diabetes or not.

However, how do we decide the value of ‘k’?

- Maybe we should use k = 1 so that we will get very good results on our training data? That might work, but we cannot guarantee that the model will perform just as well on our testing data since it can get too specific
- How about using a high value of k, say like k = 100 so that we can consider a large number of nearest points to account for the distant points as well? However, this kind of model will be too generic and we cannot be sure if it has considered all the possible contributing features correctly.

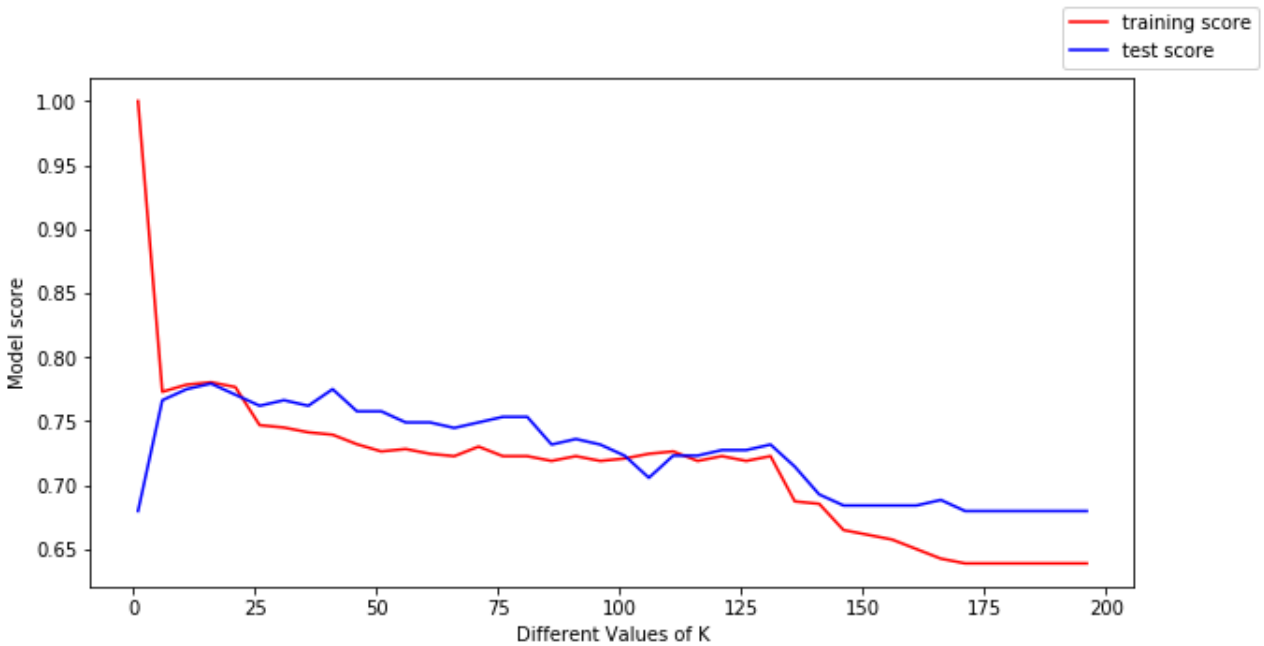
Let us take a few possible values of k and fit the model on the training data for all those values. We will also compute the training score and testing score for all those values.

```
train_score = []
test_score = []
k_vals = []

for k in range(1, 21):
    k_vals.append(k)
    knn = KNeighborsClassifier(n_neighbors = k)
    knn.fit(X_train, y_train)

    tr_score = knn.score(X_train, y_train)
    train_score.append(tr_score)

    te_score = knn.score(X_test, y_test)
    test_score.append(te_score)
```

To derive more insights from this, let us plot the training data(in red) and the testing data(in blue).

```
plt.figure(figsize=(10,5))
plt.xlabel('Different Values of K')
plt.ylabel('Model score')
plt.plot(k_vals, train_score, color = 'r', label = "training score")
plt.plot(k_vals, test_score, color = 'b', label = 'test score')
plt.legend(bbox_to_anchor=(1, 1),
```

```
            bbox_transform=plt.gcf().transFigure)
plt.show()
```



Training and testing score for different K values

To calculate the scores for a particular value of k,

```
knn = KNeighborsClassifier(n_neighbors = 14)

#Fit the model
knn.fit(X_train,y_train)

#get the score
knn.score(X_test,y_test)
```

We can make the following conclusions from the above plot:

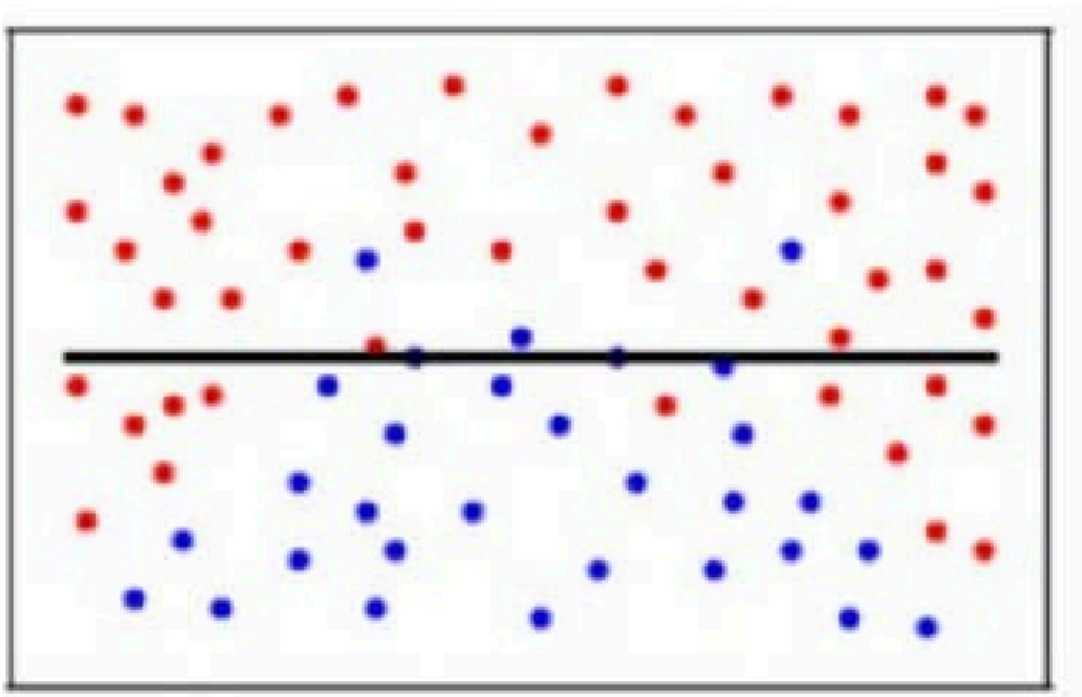
- For low values of k, the training score is high, while the testing score is low
- As the value of k increases, the testing score starts to increase and the training score starts to decrease.
- However, at some value of k, both the training score and the testing score are close to each other.

This is where Bias and Variance come into the picture.

What is Bias?

In the simplest terms, Bias is the difference between the Predicted Value and the Expected Value. To explain further, the model makes certain assumptions when it trains on the data provided. When it is introduced to the testing/validation data, these assumptions may not always be correct.

In our model, if we use a large number of nearest neighbors, the model can totally decide that some parameters are not important at all. For example, it can just consider that the Glusoce level and the Blood Pressure decide if the patient has diabetes. This model would make very strong assumptions about the other parameters not affecting the outcome. You can also think of it as a model predicting a simple relationship when the datapoints clearly indicate a more complex relationship:



simplistic model_underfitting

Mathematically, let the input variables be X and a target variable Y. We map the relationship between the two using a function f.

Therefore,

Y = f(X) + e

Here ‘e’ is the error that is normally distributed. The aim of our model f'(x) is to predict values as close to f(x) as possible. Here, the Bias of the model is:

Bias[f'(X)] = E[f'(X) – f(X)]

As I explained above, when the model makes the generalizations i.e. when there is a high bias error, it results in a very simplistic model that does not consider the variations very well. Since it does not learn the training data very well, it is called **Underfitting.**

What is a Variance?

Contrary to bias, the Variance is when the model takes into account the fluctuations in the data i.e. the noise as well. So, what happens when our model has a high variance?

The model will still consider the variance as something to learn from. That is, the model learns too much from the training data, so much so, that when confronted with new (testing) data, it is unable to predict accurately based on it.

Mathematically, the variance error in the model is:

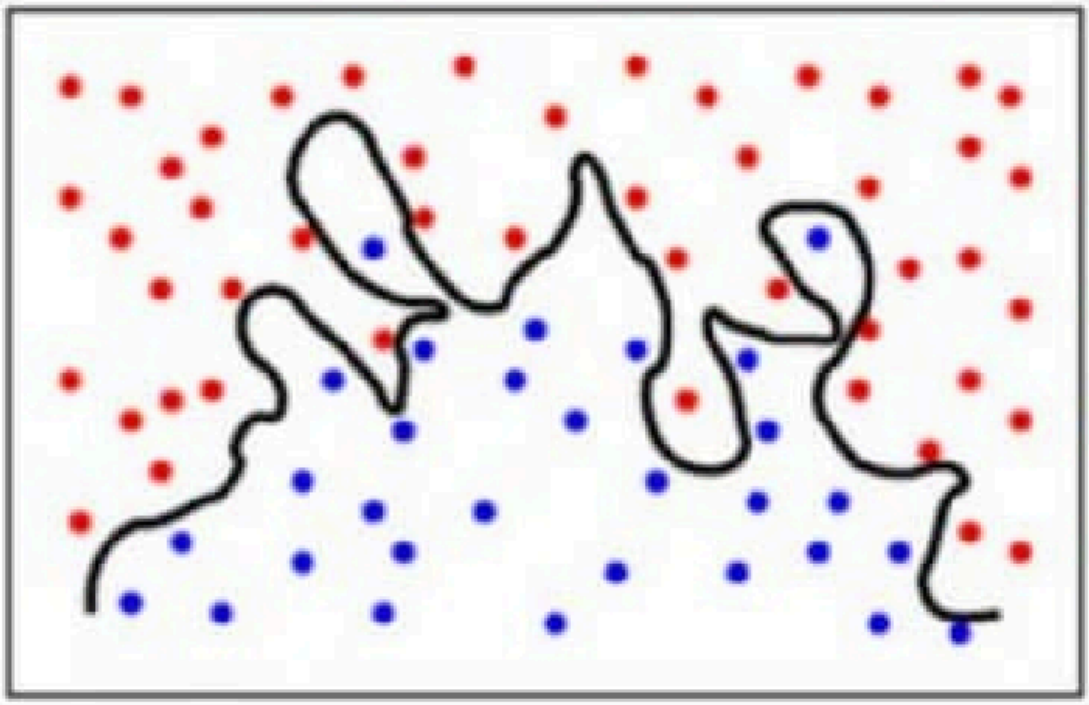
Variance[f(x) $=$

Since in the case of high variance, the model learns too much from the training data, it is called **overfitting.**

In the context of our data, if we use very few nearest neighbors, it is like saying that if the number of pregnancies is more than 3, the glucose level is more than 78, Diastolic BP is less than 98, Skin thickness is less than 23 mm and so on for every feature….. decide that the patient has diabetes. All the other patients who don't meet the above criteria are not diabetic. While this may be true for one particular patient in the training set, what if these parameters are the outliers or were even recorded incorrectly? Clearly, such a model could prove to be very costly!

Additionally, this model would have a high variance error because the predictions of the patient being diabetic or not vary greatly with the kind of training data we are providing it. So even changing the Glucose Level to 75 would result in the model predicting that the patient does not have diabetes.

To make it simpler, the model predicts very complex relationships between the outcome and the input features when a quadratic equation would have sufficed. This is how a classification model would look like when there is a high variance error/when there is **overfitting**:



Overfitting

To summarise,

- A model with a high bias error underfits data and makes very simplistic assumptions on it
- A model with a high variance error overfits the data and learns too much from it
- A good model is where both Bias and Variance errors are balanced

Checkout this article about the data science interview series

Bias-Variance Tradeoff

The bias-variance tradeoff is a fundamental concept in machine learning and statistics. It refers to the delicate balance between two sources of error in a predictive model: bias and variance.

Bias represents the error due to overly simplistic assumptions in the learning algorithm. High bias can cause the model to underfit the data, leading to poor performance on both training and unseen data.

Variance, on the other hand, reflects the model's sensitivity to small fluctuations in the training data. High variance can lead to overfitting, where the model captures noise in the training data and performs poorly on new, unseen data.

The goal is to find the right level of complexity in a model to minimize both bias and variance, achieving good generalization to new data. Balancing these factors is essential for building

models that perform well on a variety of datasets.

Understand Bias-Variance Tradeoff with the help of an example

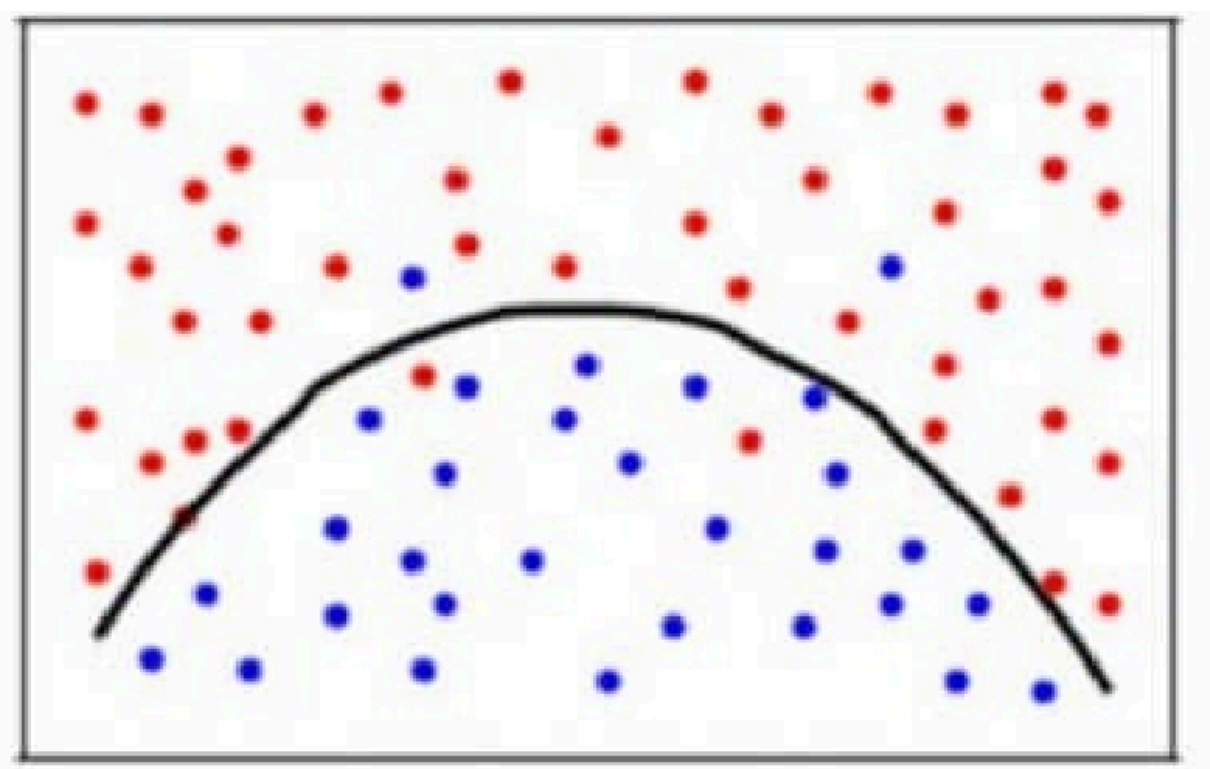
How do we relate the above concepts to our Knn model from earlier? Let's find out!

In our model, say, for, k = 1, the point closest to the datapoint in question will be considered. Here, the prediction might be accurate for that particular data point so the bias error will be less.

However, the variance error will be high since only the one nearest point is considered and this doesn't take into account the other possible points. What scenario do you think this corresponds to? Yes, you are thinking right, this means that our model is overfitting.

On the other hand, for higher values of k, many more points closer to the datapoint in question will be considered. This would result in higher bias error and underfitting since many points closer to the datapoint are considered and thus it can't learn the specifics from the training set. However, we can account for a lower variance error for the testing set which has unknown values.

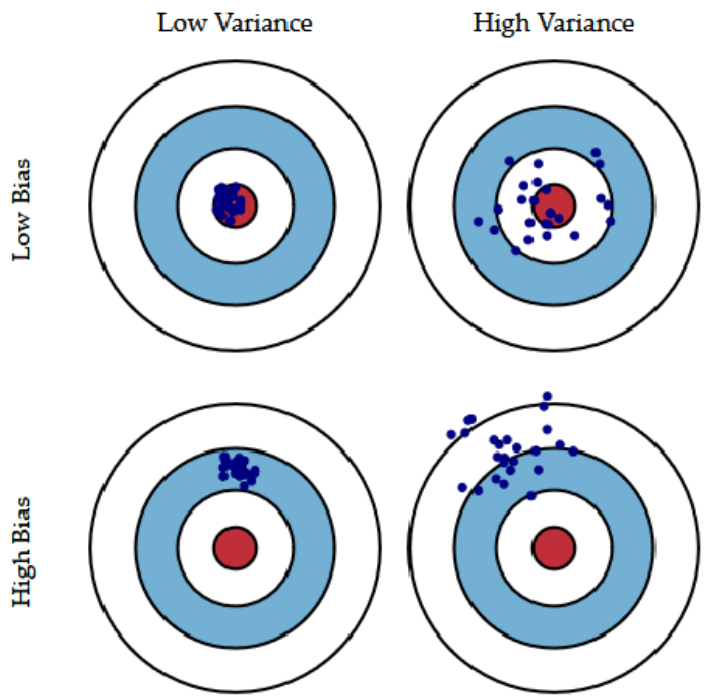
To achieve a balance between the Bias error and the Variance error, we need a value of k such that the model neither learns from the noise (overfit on data) nor makes sweeping assumptions on the data(underfit on data). To keep it simpler, a balanced model would look like this:



balanced_model

Though some points are classified incorrectly, the model generally fits most of the datapoints accurately. The balance between the Bias error and the Variance error is the **Bias-Variance Tradeoff**.

The following bulls-eye diagram explains the tradeoff better:

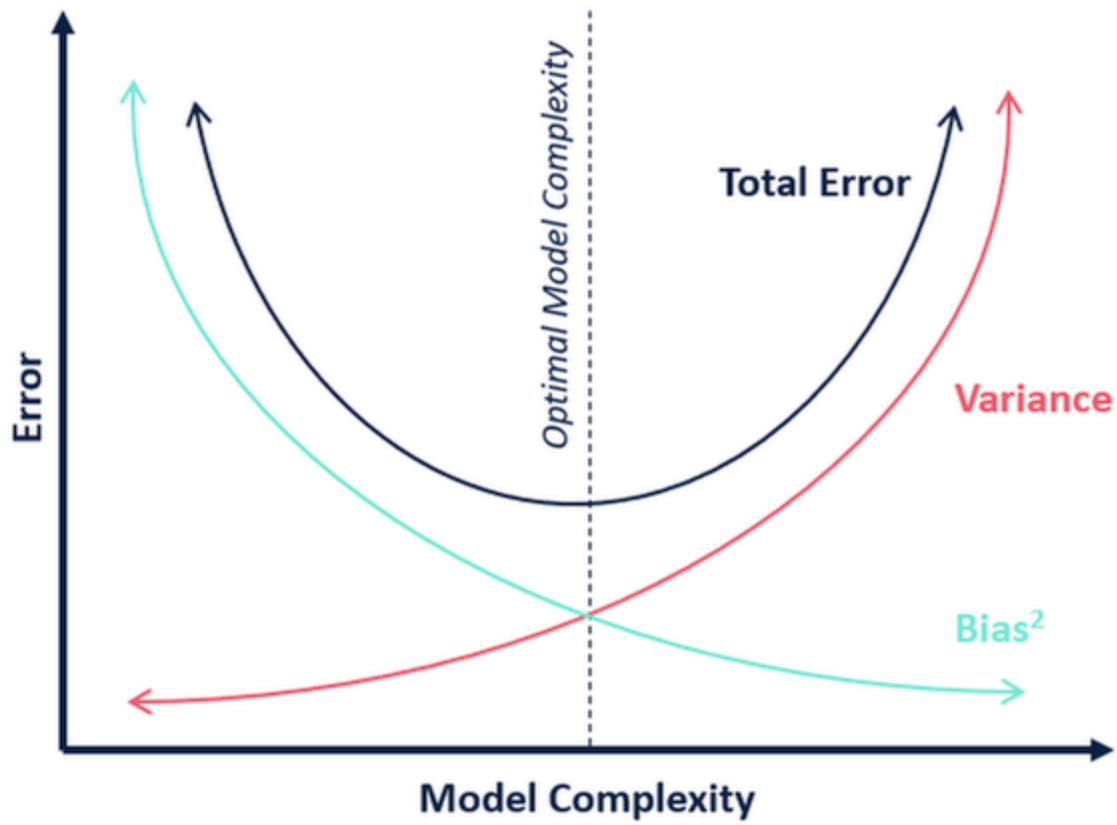


Bulls eye diagram to show bias and variance

The center i.e. the bull's eye is the model result we want to achieve that perfectly predicts all the values correctly. As we move away from the bull's eye, our model starts to make more and more wrong predictions.

A model with low bias and high variance predicts points that are around the center generally, but pretty far away from each other. A model with high bias and low variance is pretty far away from the bull's eye, but since the variance is low, the predicted points are closer to each other.

In terms of model complexity, we can use the following diagram to decide on the optimal complexity of our model.



optimal Model complexity

So, what do you think is the optimum value for k?

From the above explanation, we can conclude that the k for which

- the testing score is the highest, and
- both the test score and the training score are close to each other

is the optimal value of k. So, even though we are compromising on a lower training score, we still get a high score for our testing data which is more crucial – the test data is after all unknown data.

Let us make a table for different values of k to further prove this:

K	Training score	Testing score
k = 1	1.0	0.727
k = 13	0.782	0.779
k = 200	0.641	0.675

Bias and Variance Tradeoff: Training and testing score for different values of k

Conclusion

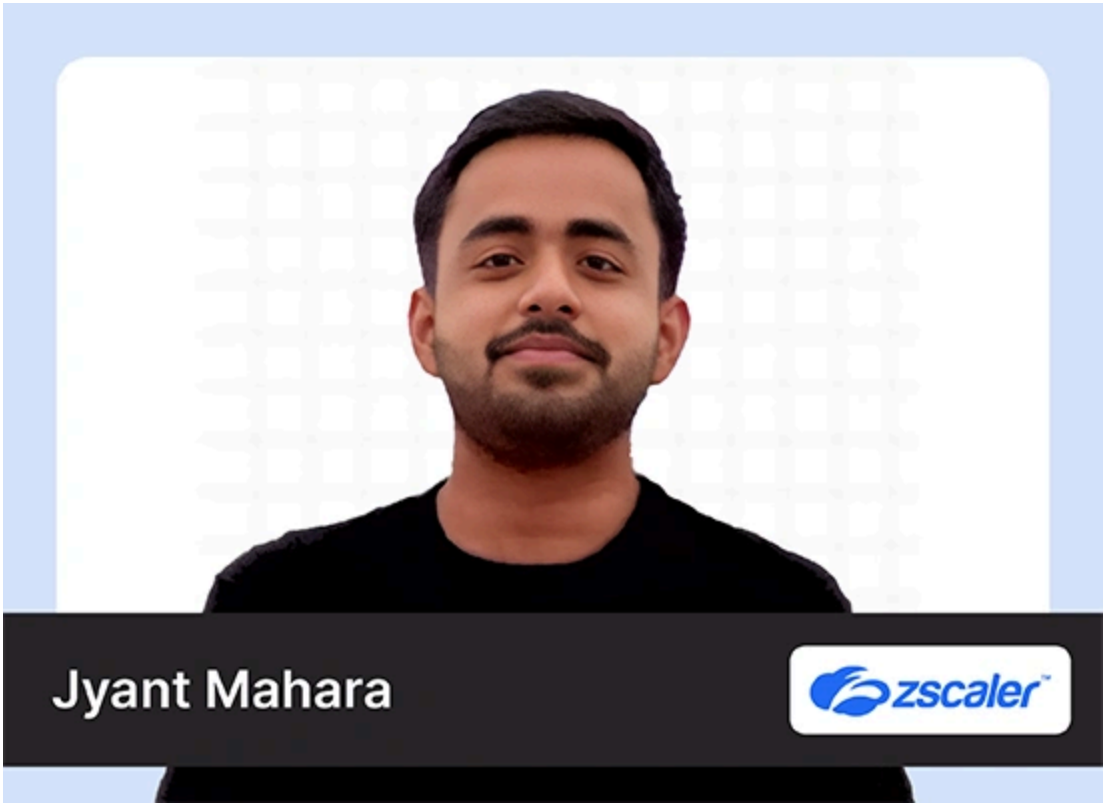
To summarize, in this article, we learned that an ideal model would be one where both the bias error and the variance error are low. However, we should always aim for a model where the model score for the training data is as close as possible to the model score for the testing data.

That's where we figured out how to choose a model that is not too complex (High variance and low bias) which would lead to overfitting and nor too simple(High Bias and low variance) which would lead to underfitting.

Bias and Variance plays an important role in deciding which predictive model to use. I hope this article explained the concept well.

Hope you like the article! The bias-variance tradeoff in machine learning is an important idea. It helps us understand the balance between bias and variance in machine learning models for better performance.

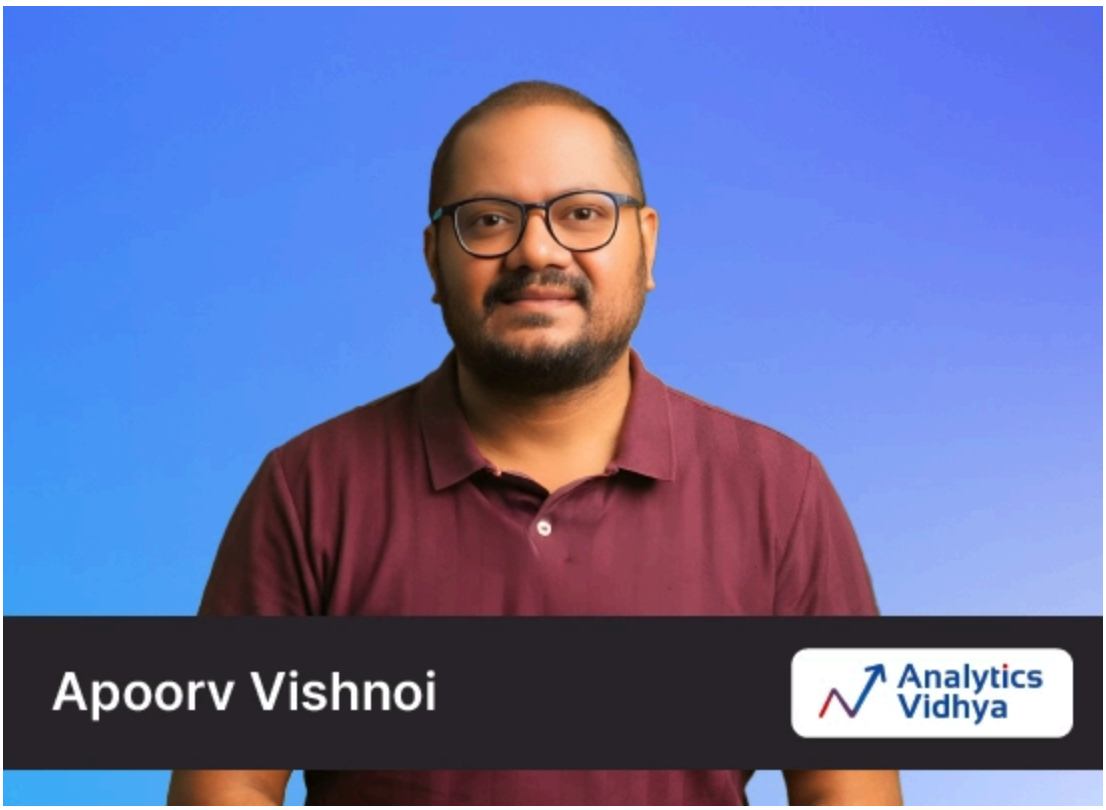
Free Courses



4.6

Soumil Jain
Analytics Vidhya

4.5


Apoorv Vishnoi
Analytics Vidhya

4.5

4.7

4.9

Recommended Articles

- GPT-4 vs. Llama 3.1 – Which Model is Better?
- Llama-3.1-Storm-8B: The 8B LLM Powerhouse Surpa...

- A Comprehensive Guide to Building Agentic RAG S...
- Top 10 Machine Learning Algorithms in 2026
- 45 Questions to Test a Data Scientist on Basics...
- 90+ Python Interview Questions and Answers (202...
- 8 Easy Ways to Access ChatGPT for Free
- Prompt Engineering: Definition, Examples, Tips...
- What is LangChain?
- What is Retrieval-Augmented Generation (RAG)?

Responses From Readers

Hi AWESOME POST!! One clarity is needed: From the bulls-eye diagram High Bias & Low Variance case, the points are away from target(Ground truth both in Training & Testing) then how by the defintion of variance (high if model is unable to predict new unseen data) its low? Kindly help me improve myself on this please.

Great explanation. I needed this. I think data normalization should be done after splitting the data and not before it because it adds is a potential bias in the evaluation of the performance. Let me know your opinion.

Hi Padma, This is a great post. I was wondering though, how the "model can totally decide that some parameters are not important at all"? I can't picture this graphically. Each plot is a record of 9 dimensions, correct? All the features are computed to make a plot for each record. So I don't see how only the "Glucose level and the Blood Pressure decide if the patient has diabetes". I can see that these two features have relatively high values, but the plots to not represent sum totals of the records, but rather a plot. So how can they influence the model so much for a high k value? For any k value, for that matter. Please help me understand this.

Frequently Asked Questions

A. The bias-variance tradeoff in machine learning involves managing two types of errors. Bias arises from overly simplistic models, leading to underfitting, while variance results from complex models capturing noise, causing overfitting. Balancing these errors is crucial for creating models that generalize well to new data, optimizing performance and robustness.

A. The bias-variance method is an approach in machine learning that analyzes the tradeoff between bias and variance to optimize model performance. By adjusting a model's complexity, it aims to strike a balance between underfitting (high bias) and overfitting (high variance). This method guides the selection of appropriate models, helping to create accurate and robust predictions on new data.

In machine learning, bias and variance are two essential concepts that influence a model's ability to generalize to unseen data. Bias represents the inherent error due to the model's assumptions, while variance measures the model's sensitivity to training data. A balance between these two error types is crucial for optimal performance.

The bias-variance tradeoff is about finding the right balance between simplicity and complexity in a machine learning model. High bias means the model is too simple and consistently misses the target, while high variance means the model is too complex and shoots all over the place. You want to aim for a model that's just right – not too simple, not too complex – to make accurate predictions on new data.